PakQuant — Qiskit Fall Fest 2026 Hackathon

Prioritizing critical infrastructure during power shortages with QAOA, validated against an exact classical baseline

Team: Muhammad Abdul Rehman (26i-0526) · Muhammad Shehryar Malik · Muhammad Taha Ikram

Institution: FAST-NUCES Islamabad

Repository: https://github.com/Abdul-Rehman-9127/Pakquant

1. Problem and who it serves

During a power shortage, a grid operator cannot supply everyone and must decide which facilities stay powered. The facilities are not equal: an ICU losing power can cost lives within minutes, while a shopping mall losing power costs only comfort. In Pakistan, where scheduled load-shedding and supply shortfalls are a routine reality, this decision is made repeatedly, often by hand and under time pressure.

The people who would use a tool like this are grid operators and distribution companies running load-shedding schedules, and emergency and disaster-response planners who must triage limited power during floods, heat waves or fuel shortages. Our scenario is deliberately small and concrete: only 3 of 5 critical institutions can be powered this hour, and the goal is to pick the 3 that minimize harm.

2. Formulation

2.1 Decision variables

$x_i \in \{0,1\}$ for $i = 1..5$, where $x_i = 1$ means institution i is kept powered.

2.2 Objective and constraint

Maximize the total lives-at-risk score of the powered institutions, subject to the supply limit of exactly 3:

$$\text{maximize} \quad \Sigma\ \text{score}_i \cdot x_i \qquad \text{subject to} \quad \Sigma\ x_i = 3$$

2.3 Test instance

Institution	Lives-at-risk score
ICU	100
Dialysis center	80
Water pumping station	40
School	5
Mall	1

The scores encode how many lives are at risk if the facility loses power. They are illustrative weights chosen to reflect that ranking, not measured data.

2.4 From constraint to QUBO to Ising Hamiltonian

QAOA minimizes the energy of a Hamiltonian, and has no native way to express “exactly 3”. We therefore fold the constraint into the objective as a quadratic penalty (a QUBO):

$$\text{cost}(x) = -\Sigma\ w_i \cdot x_i \ + \ A \cdot (\Sigma\ x_i - 3)^2 \qquad \text{with } w_i = \text{score}_i / 100$$

Substituting $x_i = (1 - Z_i)/2$ turns this into an Ising Hamiltonian made only of identity, Z and ZZ terms, which Qiskit represents as a SparsePauliOp. For $n = 5$ and $k = 3$ the coefficients are: Z_i: $w_i/2 - A(n/2 - k)$; Z_iZ_j ($i<j$): $A/2$; constant: $-\Sigma w_i/2 + A(n/2 - k)^2 + An/4$. The program verifies that the Hamiltonian's diagonal equals the QUBO cost on all 32 bitstrings before running QAOA.

Two technical choices that matter. (1) Scores are divided by 100 so the Hamiltonian has O(1) coefficients: QAOA angles are periodic, and large integer coefficients make the angle landscape oscillate so fast that the classical optimizer gets stuck. (2) The penalty $A = 1.0$ equals 100 lives, the largest single score. Any A at least as large as the biggest single score guarantees that no rule-breaking state can beat the best valid one; for this instance the true minimum is only $A > 0.05$, so 1.0 is a conservative, safe choice.

3. Why QAOA

The task is a combinatorial optimization over binary choices, which is exactly the structure QAOA targets. QAOA prepares a uniform superposition over all $2^5 = 32$ bitstrings, then alternates a cost layer (phases set by the Hamiltonian) and a mixer layer (rotations that move amplitude between bitstrings). A classical optimizer, COBYLA, tunes the 2p angles to minimize the measured energy, which shifts probability toward low-cost, i.e. high-value and valid, bitstrings.

What we claim, and what we do not. QAOA does not “check all combinations at once and read out the best”: measurement returns one bitstring per shot, and the quality of the answer depends on how well the angles concentrate probability. We make no claim of quantum advantage on this instance. At 5 institutions brute force checks 32 states instantly, and because the scores are additive, a simple sort-and-pick-top-3 also solves it exactly. The value of this project is a complete, verified pipeline (problem → QUBO → Ising → QAOA → sampling → comparison to ground truth) on a problem whose answer we know, which is the right way to test a quantum optimizer before scaling it.

Why this formulation is the right starting point for larger problems: real grid decisions are not additive. Feeder topology, line capacity, interdependence between facilities and time-varying demand turn the problem into a genuinely hard constrained combinatorial optimization, where the search space grows exponentially (2^n) and exact enumeration stops being practical. The same QUBO → Ising → QAOA pipeline extends to those cases by adding terms to the Hamiltonian.

4. Architecture

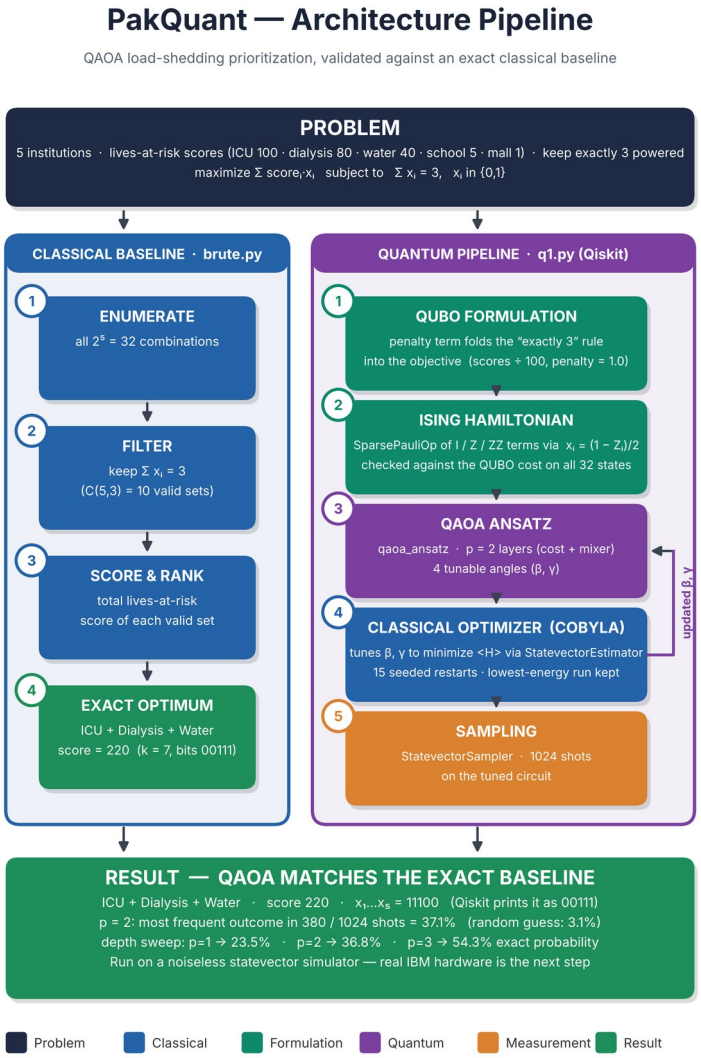


Figure 1. Both pipelines start from the same problem; the classical enumeration provides the ground truth that the quantum result is checked against.

5. Implementation

5.1 Classical baseline (brute.py)

Enumerates all $2^5 = 32$ combinations, keeps the 10 that power exactly 3 institutions, ranks them, and reports the best. Result: k = 7 (bits 00111), score 220, i.e. ICU + dialysis + water.

5.2 Quantum solver (q1.py)

- **build_hamiltonian()**: constructs the SparsePauliOp from the QUBO coefficients above.
- **Sanity check**: asserts the Hamiltonian diagonal equals cost(x) for all 32 bitstrings.
- **run_qaoa(H, p, seed)**: builds qaoa_ansatz(H, reps=p) and minimizes ⟨H⟩ with SciPy COBYLA through Qiskit's StatevectorEstimator, using 15 seeded random restarts and keeping the run with the lowest energy (selection never uses knowledge of the answer).
- **Sampling and decoding**: the tuned circuit is sampled 1024 times with StatevectorSampler (fixed seed), and each bitstring is decoded to institution names, score and validity.

Bit-order convention. Qubit i ↔ institution i (0 = ICU … 4 = mall). Qiskit prints bitstrings as q4…q0, so the optimal answer ICU + dialysis + water appears as 00111 in both brute.py and Qiskit's output; in institution order $x_1 \ldots x_5$ it is 11100. Both forms are printed side by side to avoid confusion.

6. Results against the exact baseline

Method	Best answer	Score	Notes
Brute force	ICU + dialysis + water	220	Exact, 32 combinations checked
QAOA (p=2)	ICU + dialysis + water	220	Most frequent outcome: 380 / 1024 shots = 37.1%

Top outcomes of the 1024-shot run (p = 2):

Qiskit string	$x_1 \ldots x_5$	Shots	Powered	Status
00111	**11100**	**380 (37.1%)**	**ICU + dialysis + water**	**Valid, score 220 (optimal)**
00011	11000	110 (10.7%)	ICU + dialysis	Invalid: only 2 powered
01111	11110	90 (8.8%)	ICU + dialysis + water + school	Invalid: 4 powered
10111	11101	77 (7.5%)	ICU + dialysis + water + mall	Invalid: 4 powered
01110	01110	56 (5.5%)	dialysis + water + school	Valid, score 125
01011	11010	55 (5.4%)	ICU + dialysis + school	Valid, score 185

Reading the result honestly. The optimal answer is the single most likely outcome at 37.1% of shots (exact probability 36.8%), about 12× better than random guessing over 32 states (3.1%) and 3.7× better than picking uniformly among the 10 valid sets (10%). 60.1% of shots were valid (exactly 3 powered); the rest violate the supply limit, which is expected with a penalty-based encoding and the standard mixer. Final energy $\langle H \rangle = -1.413$ (normalized units); best angles $\beta = [-0.516, 1.158]$, $\gamma = [3.077, -0.391]$ (qaoa_ansatz orders its parameters β first, then γ).

Effect of circuit depth

p	Energy ⟨H⟩	P(optimal answer)	P(valid)
1	−1.054	23.5%	53.1%
2	**−1.413**	**36.8%**	**60.3%**
3	−1.717	54.3%	69.1%

Deeper circuits lower the energy and raise the probability of the optimal answer, as QAOA theory predicts, while adding 2 angles per layer for COBYLA to tune. Probabilities in this table are exact statevector values; the sampled headline run is within 0.3 percentage points of the p=2 value. The p=3 row can differ in the last digit across Qiskit and SciPy versions.

A bug we caught, and what it taught us

Our first version used raw coefficients with a penalty of 500. Its most frequent output, 11100 in Qiskit order, looked like “the answer” if the bitstring was read in reverse. Decoding it properly showed that it was in fact water + school + mall (score 46), and the exact probability of the true optimum was only about 0.2%. The sanity check had been proving the Hamiltonian correct all along; the weakness was the badly scaled landscape plus a bit-order misreading. Normalizing the scores,

using a principled penalty, adding restarts, and decoding every bitstring to institution names fixed it. We kept the decoding and validity reporting in the code so this class of error cannot go unnoticed.

7. Limitations

- **No quantum advantage claimed.** With 5 additive-score institutions the problem is trivially solvable classically (even by sorting). Our contribution is a verified end-to-end pipeline, not a speed-up.
- **Probabilistic output.** The right answer is found in about 1 of 3 shots at p=2, and 40% of shots violate the supply limit. In practice one samples many shots, discards invalid ones, and picks the best valid bitstring by score; a constraint-preserving mixer would raise validity further.
- **Simulator only.** Results come from Qiskit's StatevectorEstimator and StatevectorSampler: exact, noiseless evolution. On real hardware, gate errors, decoherence and readout noise (the NISQ regime) would lower success rates; this project does not yet test that.
- **Scaling.** The search space doubles with every added facility ($2^{50} \approx 10^{15}$ at 50), but we have not shown QAOA beating good classical heuristics at any size. Illustrative scores are weights, not real casualty data.

8. Next steps

- Run the same circuit on real IBM Quantum hardware through Qiskit Runtime and compare the noisy success rate with the simulator's 37.1%.
- Add a noise model (Qiskit Aer) as an intermediate step, and try a constraint-preserving (Dicke / XY) mixer to raise the valid-shot fraction.
- Extend to larger, non-additive instances (feeder topology, capacity limits) where exact enumeration is no longer a realistic baseline.

9. Reproducibility

Python 3.10+ with qiskit, numpy, scipy (see requirements.txt). Run python brute.py, then python q1.py (a minute or two on a laptop). Random seeds are fixed, so the sampled counts reproduce on the same Qiskit version (tested with Qiskit 2.6.2). An independent NumPy re-implementation of the same QAOA (verify_numpy.py, included in the repository) reproduces the p = 1 and p = 2 results exactly.

10. Credits

- Built using Qiskit (IBM open-source quantum SDK).
- Classical optimization via SciPy (COBYLA).
- AI assistance (Claude) was used for explanation, debugging and review while writing and understanding the code and this report. All conclusions and results are the team's own.